

نقشه راه ۶ ماهه شخصی

از QA Lead به مهندس نرم افزار با ذهنیت کیفیت

زبان: Python (ادامه pytest موجود)

زمان: ۱۰ ساعت در هفته ۲۶۰۰ ساعت

پروفايل: QA Lead · تيم ۶ نفره · JMeter · Playwright · Postman

تاريخ تهيه: تير ۱۴۰۵ / ژوئيه ۲۰۲۶

جمع بندی ارزیابی

تو تستر حرفه ای هستی که باید بشه مهندس با ذهنیت تست. از صفر شروع نمی کنی — ۱۴ سال تجربه QA مزیت توست.

Skill Matrix — سطح واقعی

حوزه	نمره	وضعیت
Test Strategy & Test Case	★★★★★	قوی — مزیت رقابتی
API Testing (Postman)	★★★★★	حرفه ای
Test Automation (Playwright/Selenium)	★★★★	می دانی، تنها انجام می دهی
Performance Testing (JMeter)	★★★★	تجربه داری
CI/CD	★★★★	استفاده کردی، عمیق نیست
Git	★★★	توی کار می زنی، پایه سوراخ دارد
SQL	★★★★	JOIN بلدی
Programming (Python/JS)	★★★	script می نویسی، نه engineer
Docker	★	فقط شنیدی
CS Concepts	★★★	HTTP خوب، race condition نمی دانی
Debugging / Logs	★★★	گاهی می فهمی
System Concepts	★★★	شنیدی، عمیق نیست

فاصله با مهندس جهانی

تو الان:	QA Lead قوی
هدف ۶ ماه:	SWE + QA Engineer
هدف ۲ سال:	Junior Backend SWE
هدف ۵ سال:	Mid/Senior SWE

چیزهایی که داری (مهندس تازه کار ندارد)

- Test strategy و test case حرفه ای
- Postman, Playwright, JMeter
- درک flaky test, staging vs production
- Python در pytest و JS در Postman

چیزهایی که باید بسازی

- Git واقعی (نه فقط pull)
- Docker عملی
- برنامه نویسی engineer-level

- مفاهیم: consumer, backward compatible, race condition
- خواندن log و debug

اصلاح مفاهیم HTTP

کد	معنی درست	اشتباه رایج
404	منبع پیدا نشد (Not Found)	❌ دسترسی نداری
403	دسترسی ممنوع (Forbidden)	—
401	احراز هویت نشده (Unauthorized)	—
500	خطای سرور (Internal Server Error)	✓ درست می دانی

مفاهیم کلیدی — توضیح ساده

Git — دفترچه تغییرات تیم

هر تغییر کد = commit. هر PR = branch = feature = درخواست merge.

```
git add نام-فایل
git commit -m "اضافه کردن تست لاگین"
git push
```

Docker — جعبه آماده اجرا

برنامه + همه dependencyها در یک container. روی لپتاپ و سرور یکسان اجرا می شود.

Backward Compatible — تغییر جدید، قدیمی را نشکند

API نسخه ۲ فیلد جدید اضافه می کند ولی فیلد قدیمی را حذف نمی کند. اپ قدیمی هنوز کار می کند.
سوال در جلسه: «این تغییر backward compatible هست؟»

Consumer / Producer

Producer پیام می فرستد → [صف پیام] → Consumer پیام را می گیرد و واکنش نشان می دهد.
سوالات تست: اگر پیام دوبار برسد؟ اگر consumer خاموش باشد؟

Race Condition

دو عملیات همزمان روی یک منبع → نتیجه غلط. مثال: دو نفر همزمان یک صندلی رزرو کنند و هر دو موفق شوند.

نقشه ۶ ماهه — هفته به هفته

Git + Docker ماه ۱

چرا اول؟ فردا در کار به کارت می‌آید. ضعیف‌ترین بخش فعلی تو.

هفته ۱-۲: Git

روز	کار (۱.۵ ساعت)
۱	نصب git config — Git — اولین git init
۲	git add → commit → status
۳	git log → git diff روی repo کاری
۴	branch — checkout → git branch جدید
۵	git merge — conflict بساز و حل کن
۶	Pull Request روی GitHub
۷	مرور + یادداشت

هفته ۲-۳: Docker

- docker run hello-world
 - docker run postgres — دیتابیس بالا بیاور
 - docker-compose.yml بنویس
 - محیط تست محلی با Docker
- خروجی: ۵ commit روی GitHub + docker-compose با PostgreSQL

ماه ۲ مفاهیم سیستمی + API عمیق

هفته ۵-۶: API عمیق‌تر

- REST: headers, query params, body types
 - API versioning: /v1/users vs /v2/users
 - contract testing و Backward compatible
- تمرین: API واقعی تست کن — کدام تغییر breaking است؟

هفته ۷-۸: Messaging + Race Condition

Consumer/Producer, سوالات تست async, یک flaky test واقعی تحلیل کن.

خروجی: سند ۱ صفحه‌ای «Backward compatibility در پروژه ما»

Python Engineer (از pytest)

ماه ۳

هفته ۹-۱۰: **Python** محکم

Functions, classes, file I/O, error handling, venv + requirements.txt

هفته ۱۱-۱۲: **FastAPI** — اولین API

```
GET /tasks      → لیست
POST /tasks     → ایجاد
GET /tasks/{id} → یک task
```

تست با httpx + pytest

خروجی: API روی GitHub با تست های pytest

CI/CD + Log + Debug **ماه ۴****هفته ۱۲-۱۴: GitHub Actions**

commit → lint → test → build — هر push تست خودکار

هفته ۱۵-۱۶: Log خواندن

- Log levels: DEBUG, INFO, WARNING, ERROR
- grep / tail -f
- trace یک باگ واقعی از کار

خروجی: pipeline سبز + توانایی خواندن log

SQL + Database **ماه ۵****هفته ۱۷-۱۸: PostgreSQL + Python**

API را به PostgreSQL وصل کن — migration backward compatible

هفته ۱۹-۲۰: Query Optimization

EXPLAIN ANALYZE — query کند پیدا و بهینه کن

خروجی: API با database واقعی

پروژه جمع بندی **ماه ۶****هفته ۲۱-۲۶: Task Manager Engineer Edition**

FastAPI + PostgreSQL ☐

CRUD کامل ☐

pytest — حداقل ۲۰ تست (unit + integration) ☐

Docker + docker-compose ☐

GitHub Actions CI ☐

README انگلیسی ☐

یک ADR: «چرا این طراحی؟» ☐

این پروژه = رزومه ۶ ماه بعد.

تقسیم ۱۰ ساعت هفتگی

فعالیت	ساعت
یادگیری (یک مفهوم)	۳
دست به کار (تمرین/پروژه)	۴
انگلیسی (مستندات همان موضوع)	۲
یادداشت: «امروز چی یاد گرفتم؟»	۱

هفته ۱ — شروع از فردا

روز	دقیقاً چه کار کن
شنبه	Git نصب — git config --global user.name و user.email
یکشنبه	پوشه بساز — git init — README.md — git add — git commit
دوشنبه	repo کاری: git log --oneline -10
سه‌شنبه	checkout — git branch test-branch — تغییر — commit
چهارشنبه	git merge test-branch
پنج‌شنبه	repo — GitHub عمومی — push
جمعه	یادداشت: «Git یعنی چی و QA Lead چرا باید بلد باشد؟»

قبل و بعد — ۶ ماه

الان	۶ ماه بعد
Git می‌زنم ولی commit بلد نیستم	Git workflow راحت
Docker شنیدم	محیط تست با Docker بالا می‌آورم
Consumer چیه؟	سوال درست در جلسه می‌پرسم
pytest می‌نویسم	API + تست + CI خودم ساختم
Log گاهی می‌فهمم	مسیر debug را می‌دانم

نقش AI — قانون طلایی

غلط ❌	درست ✅
«کل پروژه را بنویس»	«Docker چیه؟ مثل چی می‌ماند؟»
«تست‌هایم را خودت بنویس»	«این خطا یعنی چی؟»
«به‌جای من یاد بگیر»	«کدام را review کن»

قانون: اگر نتوانی بدون AI همان کار را تکرار کنی → هنوز یاد نگرفته‌ای.
قانون کار: هر چیزی یاد گرفتی، فردا در کار امتحانش کن.

چک‌لیست پایان ۶ ماه

- ☐ + commit واقعی روی GitHub
- ☐ docker-compose کار می‌کند

☐ یک API با FastAPI نوشتم☐ +۲۰ تست pytest☐ CI pipeline سبز☐ README انگلیسی☐ می فهمم backward compatible یعنی چی☐ می فهمم consumer/producer یعنی چی☐ می توانم log بخوانم

نقشه راه ۶ ماهه شخصی — QA Lead به مهندس نرم افزار Python ۰۱۰۰ ساعت/هفته ۲۰۲۶۰